Rheumatoid Arthritis Image Segmentation/Classification

1. Installing dependencies.
2. Loading data from Roboflow.
3. Building/Modifying a model.
4. Training with different learning rates and visualization.

```
!pip install -q roboflow ultralytics
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 184.0/184.0 kB 8.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 66.8/66.8 kB 5.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 49.9/49.9 MB 16.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 47.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 56.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.5/5.5 MB 83.8 MB/s eta 0:00:00
```

1. Load Data from Roboflow

Replace the placeholders with actual Roboflow project details.

```
from roboflow import Roboflow

# # Initialize Roboflow (Uncomment and fill in your details)
# rf = Roboflow(api_key="YOUR_API_KEY")
# project = rf.workspace("YOUR_WORKSPACE").project("YOUR_PROJECT")
# dataset = project.version(1).download("yolov8") # or appropriate format
```

2. Model Development & Layer Modification

A function to build the model so can easily modify the layers and change learning rates.

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

def build_custom_model(input_shape, learning_rate=0.001):
    model = models.Sequential([
        layers.Input(shape=input_shape),
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.2),
        layers.Dense(32, activation='relu'),
        layers.Flatten(),
        layers.Dense(1, activation='sigmoid')
    ])

    optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss='binary_crossentropy', metrics=['accuracy'])
    return model

# Using the existing data from kernel state
lr_list = [0.01, 0.001, 0.0001]
histories = []

for lr in lr_list:
```

```
    print(f"Training with learning rate: {lr}")
    model = build_custom_model(input_shape=X_train.shape[1:], learning_rate=lr)
    # Training for 20 epochs to see convergence
    history = model.fit(X_train, y_train, epochs=20, validation_data=(X_test, y_test), verbose=0)
    histories.append((lr, history))

print("Training complete. You can now run the visualization cell.")
```

```
Training with learning rate: 0.01
Training with learning rate: 0.001
Training with learning rate: 0.0001
Training complete. You can now run the visualization cell.
```

3. Visualization

Compare the loss and accuracy across the different learning rates tested above.

4. Image Segmentation with Custom U-Net

The implementation of the U-Net architecture from scratch using TensorFlow/Keras.

```
def conv_block(inputs, num_filters):
    x = layers.Conv2D(num_filters, 3, padding='same')(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Conv2D(num_filters, 3, padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    return x

def encoder_block(inputs, num_filters):
    x = conv_block(inputs, num_filters)
    p = layers.MaxPooling2D((2, 2))(x)
    return x, p

def decoder_block(inputs, skip_features, num_filters):
    x = layers.Conv2DTranspose(num_filters, (2, 2), strides=2, padding='same')(inputs)
    x = layers.Concatenate()([x, skip_features])
    x = conv_block(x, num_filters)
    return x

def build_unet(input_shape):
    inputs = layers.Input(input_shape)

    # Encoder (Contracting Path)
    s1, p1 = encoder_block(inputs, 64)
    s2, p2 = encoder_block(p1, 128)
    s3, p3 = encoder_block(p2, 256)
    s4, p4 = encoder_block(p3, 512)

    # Bottleneck
    b1 = conv_block(p4, 1024)

    # Decoder (Expansive Path)
    d1 = decoder_block(b1, s4, 512)
    d2 = decoder_block(d1, s3, 256)
    d3 = decoder_block(d2, s2, 128)
    d4 = decoder_block(d3, s1, 64)

    outputs = layers.Conv2D(1, 1, padding='same', activation='sigmoid')(d4)
```

```
    model = models.Model(inputs, outputs, name='U-Net')
    return model

# Initialize the U-Net model
# Assuming image size 128x128 based on previous code
img_shape = (128, 128, 1)
unet_model = build_unet(img_shape)
unet_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
unet_model.summary()
```

Model: "U-Net"

Layer (type)	Output Shape	Param #	Connected to
input_layer_10 (InputLayer)	(None, 128, 128, 1)	0	-
conv2d_13 (Conv2D)	(None, 128, 128, 64)	640	input_layer_10[0…
batch_normalization (BatchNormalizatio…	(None, 128, 128, 64)	256	conv2d_13[0][0]
activation (Activation)	(None, 128, 128, 64)	0	batch_normalizat…
conv2d_14 (Conv2D)	(None, 128, 128, 64)	36,928	activation[0][0]
batch_normalizatio… (BatchNormalizatio…	(None, 128, 128, 64)	256	conv2d_14[0][0]
activation_1 (Activation)	(None, 128, 128, 64)	0	batch_normalizat…
max_pooling2d_6 (MaxPooling2D)	(None, 64, 64, 64)	0	activation_1[0][…
conv2d_15 (Conv2D)	(None, 64, 64, 128)	73,856	max_pooling2d_6[…
batch_normalizatio… (BatchNormalizatio…	(None, 64, 64, 128)	512	conv2d_15[0][0]
activation_2 (Activation)	(None, 64, 64, 128)	0	batch_normalizat…
conv2d_16 (Conv2D)	(None, 64, 64, 128)	147,584	activation_2[0][…
batch_normalizatio… (BatchNormalizatio…	(None, 64, 64, 128)	512	conv2d_16[0][0]
activation_3 (Activation)	(None, 64, 64, 128)	0	batch_normalizat…
max_pooling2d_7 (MaxPooling2D)	(None, 32, 32, 128)	0	activation_3[0][…
conv2d_17 (Conv2D)	(None, 32, 32, 256)	295,168	max_pooling2d_7[…
batch_normalizatio… (BatchNormalizatio…	(None, 32, 32, 256)	1,024	conv2d_17[0][0]
activation_4 (Activation)	(None, 32, 32, 256)	0	batch_normalizat…
conv2d_18 (Conv2D)	(None, 32, 32,	590,080	activation_4[0][…

Visualize Segmentation Results

Compare the original image, the ground truth mask, and the predicted mask.

```
import numpy as np

def display_segmentation(display_list):
    plt.figure(figsize=(15, 15))
    title = ['Input Image', 'True Mask', 'Predicted Mask']
    for i in range(len(display_list)):
        plt.subplot(1, len(display_list), i+1)
        plt.title(title[i])
        plt.imshow(tf.keras.utils.array_to_img(display_list[i]))
        plt.axis('off')
    plt.show()

# Example visualization logic (requires running model.predict on a sample)
# sample_image = x_test[0]
# sample_mask = y_test[0]
# pred_mask = unet_model.predict(sample_image[tf.newaxis, ...])[0]
# display_segmentation([sample_image, sample_mask, pred_mask])
```

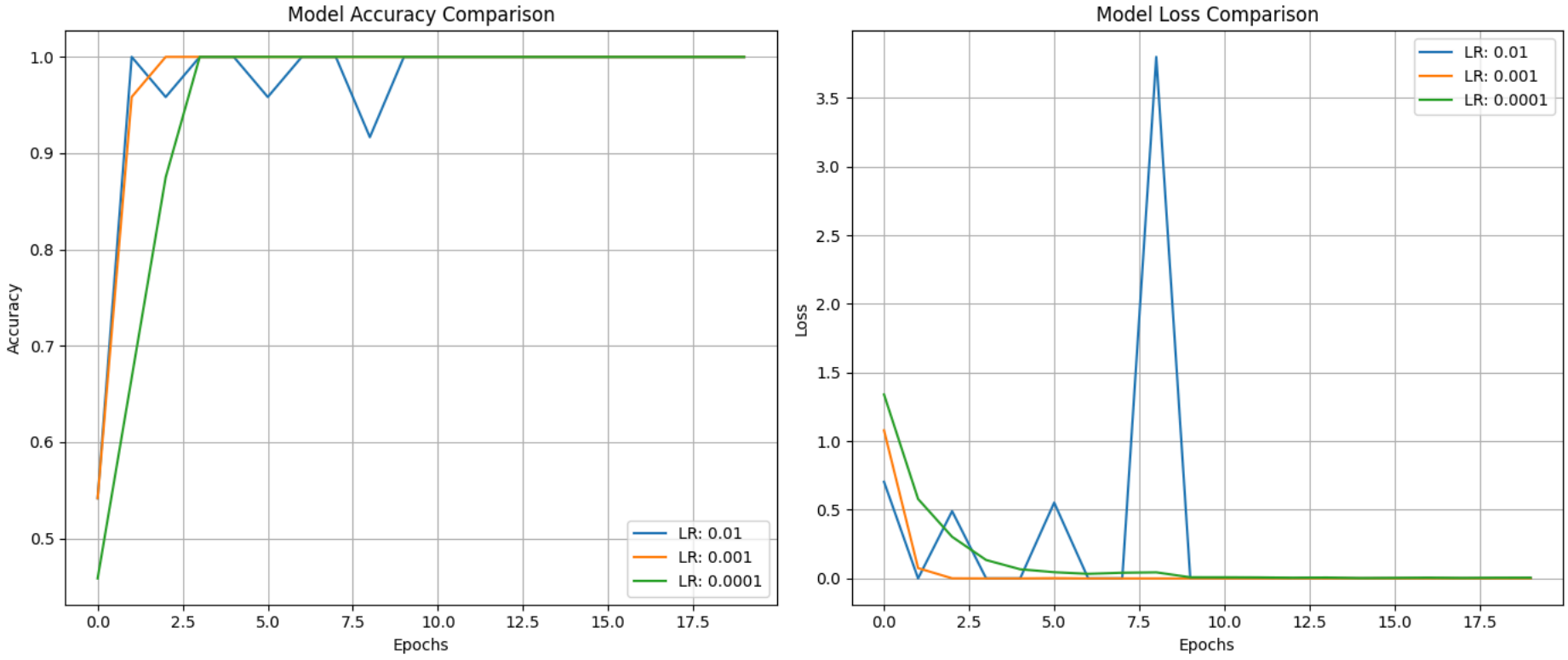
```
def plot_results(histories):
    plt.figure(figsize=(14, 6))

    # Plot Accuracy
    plt.subplot(1, 2, 1)
    for lr, history in histories:
        plt.plot(history.history['accuracy'], label=f'LR: {lr}')
    plt.title('Model Accuracy Comparison')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.grid(True)

    # Plot Loss
    plt.subplot(1, 2, 2)
    for lr, history in histories:
        plt.plot(history.history['loss'], label=f'LR: {lr}')
    plt.title('Model Loss Comparison')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.show()

# Visualize the results
plot_results(histories)
```



3. Visualization

After training, compare the loss and accuracy across different learning rates.

```
def plot_results(histories):
    plt.figure(figsize=(12, 5))

    # Plot Accuracy
    plt.subplot(1, 2, 1)
    for lr, history in histories:
        plt.plot(history.history['accuracy'], label=f'LR: {lr}')
    plt.title('Model Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
```